

Data 09 de abril de 2026

O projeto proposto é o desenvolvimento de um jogo CLI de forca em bash, visando pratica e aprofundamento no uso de Shell Script e terminal linux. Os principais comandos e tecnologias utilizadas: * \$echo para output no terminal, * \$RANDOM para escolha aleatoria de palavras * While para o controle de sessão principal, cada rodada é feita dentro de um while * shopt -S para o set de arquivos recursivamente

Como foi construido

Como base eu utilizei o projeto Termonal[<https://github.com/viniciuskant/termonal/tree/main>], que é uma implementação do jogo de adivinhação de palavras Termo para linha de comando. O projeto foi minha principal fonte de palavras em arquivos .txt e como principal estrategia eu listei um numero x de palavras por sessao aleatoriamente, isso foi definido no esboço do projeto junto do que deveria ser o frontend em um file separado, ainda sobre esse esboço eu defini que a sessao seria dentro de um while e que coisas como definir qual palavra seria escolhida e sua divisao em letras seria feita pre-carregadas.

A montagem do DB de palavras se resume em usar o shopt -S globstar puxando todos os txt para uma var de arquivos e construir o db com um indice randomico de 10 palavras, depois fazer o split de uma entre as 10 definida também com \$RANDOM para termos a palavra da vez.

Como foram separadas as funções

Aqui foi onde eu gastei um tempo refazendo, porque minha definição de qual parte do codigo merecia ter uma função própria foi refeita algumas vezes, por exemplo o \$desenhar_header faria apenas a introdução do jogo e os updates seriam feitos por outra função que chamaria a \$desenhar_hader quantas vezes fossem necessarias.

\$desenhar_header() é responsavel por fazer o frontend e atualizar visualmente o andamento ta partida. Aqui eu apanhei bastante porque eu não consegui passar direito os novos valores quando o jogador acertava as letras, atualizar os arrays era ok mas atualizar os underlines conforme ele acertava deu um trabalho, isso inclusive me fez refazer o gerar_progresso.

\$gerar_progresso() esta la para atualizar os arrays de letras certas e erradas conforme a comparação com o array da palavra da vez.

_chute() É uma função para *All win* onde o jogador pode gastar todos os trials e dizer qual é a palavra. Funciona dependente da dica (uma das ultimas a serem desenvolvidas) porque fazer o set de dicas para files de palavras foi volumoso e demorado, para o meu azar o dict - Dicionario CLI, só funciona bem em ingles então todas as palavras precisaram de uma linha em outro file correspondente com as dicas.

_tentativa() faz a tratativa de entrada, nele tem tratamento para input apenas de letras, verificação de tentativas repetidas, letras unicas e por fim se a letra está na palavra ou não. Aqui eu apanhei para definir os retornos, durante um tempo voce só sairia do jogo se forçasse um

chute, como saída eu utilizei numeros diferentes para cada estagio onde teria uma verificação e dentro do while principal defini o valor de return que deveria “quebrar” a sessão com -eq.

Melhorias

Dicas de palavra, isso deve reutilizar a estrutura atual mas vai acrescentar o trabalho de relacionar cada palavra ao indice da sua dica correspondente. Deve ter jeito melhor de fazer.

Rerun, eu não fiz um bom rerun, ele no maximo vai rodar tudo de novo sem evitar bem que a mesma sessão seja repetida logo na sequencia por exemplo.

Perguntas de Verificação (I'm not a robot?)

P1- O esboço foi definido com preciso de um DB de palavras partindo de um file ou do dict que eu sabia que poderia usar para me retornar uma palavra com alguns parametros. O que mudou muito foi a estrutura do codigo em sí, tinha menos funções e o set da palavra era feito dentro do while a principio. Acho que a principal motivação foi eu ter jogado o “frontend” para outro file então eu tinha mais espaço para organizar as coisas fora do meu main e botar pra rodar no while (eu puxei o front de volta par um file só na revisão para me enquadrar na regra do file unico)

P2-

* Busca recursiva de files em um dir com globstar e filtro pela extensão do file, todos para o array de arquivos. * Escolha de um file com \$RANDOM que gera apenas um int aleatório a principio, mas com o uso do modulo eu posso limitar o escopo do random para algo valido dividindo esse numero aleatorio por um valor fixo e usando o resto dessa divisão como o meu numero escolhido.

P3- ## While e if

```
while ((tentativas_restantes>0)); do
    progresso=$(gerar_progresso)
    desenhar_header "$tentativas_restantes" "$progresso" "${letras_certas[*]}"
    "${letras_erradas[*]}"
    read -p 'Digite uma letra: ' tentativa
    if [[ "$tentativa" == "!" ]]; then
        processar_chute
        resultado=$?
        if [[ $resultado -eq 0 ]]; then
            break
        else
            tentativas_restantes=0
            break
        fi
    else
        processar_tentativa "$tentativa"
        resultado=$?
        if [[ $resultado -eq 0 ]]; then
            letras_certas+=("$tentativa")
        elif [[ $resultado -eq 1 ]]; then
            letras_erradas+=("$tentativa")
            ((tentativas_restantes--))
        fi
    fi
fi
```

```

        progresso=$(gerar_progresso)
        #desenhar_header "$tentativas_restantes" "$progresso"
        "${letras_certas[*]}" "${letras_erradas[*]}"
        if [[ ! "$progresso" =~ "_" ]]; then
            echo " Você venceu! Palavra: $palavra"
            break
        fi
    done

```

Ele faz o chamado de todas as funções e o jogo mesmo rola aqui, enquanto o player tem tentativas restantes o jogo rola. Provavelmente um switch case seria mais clean mas como eu já tinha pré-definido para os breaks um numero pequeno de saidas eu fui de if aninhado mesmo.

tem as saidas definidas nos ifs e elifs msa no processar chute também por exemplo, onde se o input for igual a palavra (input tratado com o | tr inclusive) valida e encerra a partida. se a função retorna a em \$? tem um if para cada caso, if para ! que chama o processa chute, um que derruba a sessão se for zero e outro que chama o processa tentativas e dentro dessa função direciona o acerto ou erro da letra em um array que vai para o placar.

```

processar_chute() {
    read -p 'Digite a palavra completa: ' chute
    chute=$(echo "$chute" | tr '[:upper:]' '[:lower:]')
    if [[ "$chute" == "$palavra" ]]; then
        desenhar_header "$tentativas_restantes" "$palavra"
        "${letras_certas[*]}" "${letras_erradas[*]}"
        echo " Você acertou a palavra inteira!"
        return 0
    else
        echo " Errou o chute! Fim de jogo."
        return 1
    fi
}

```

For

```

gerar_progresso() {
    local progresso=""
    for letra in "${palavra_split[@]}; do
        if [[ " ${letras_certas[*]} " =~ " ${letra} " ]]; then
            progresso+=" $letra "
        else
            progresso+=" _ "
        fi
    done
    echo "$progresso"
}

```

aqui usei o for para atualizar o placar de letras lá na header, ele faz a verificação que tem no if passar por todas as letras da palavra que dividimos.

P5

Definir o que interfere no placar e como atualizar ele foi trabalhoso, de primeira eu iria chamar os processadores de tentativa e direcionar um return em cada para mandar direto para o placar,

mas agrupar isso em uma unica função que verifica o status da palavra com o input direto foi melhor, eu tenho uma area menor para lidar, um array a mais apenas.

O que aprendi com? principalmente sintaxe e o uso de funções em shell script, até então eu só utilizava para iniciar execução de um app ou de um comando ou outro no terminal.

dump

```
#!/bin/bash
```

```
preset_jogo() {
    db_palavras=()
    tentativas=()
    tentativas_restantes=6
    letras_certas=()
    letras_erradas=()
    mensagem="Bora começar! Digite uma letra."

    SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
    LISTAS_DIR="$SCRIPT_DIR/listas"

    shopt -s globstar
    arquivos=("$LISTAS_DIR/**/*.txt)
#random para pegar o file
    indice=$((RANDOM % ${#arquivos[@]}))
    arquivo_escolhido=${arquivos[$indice]}
# DB de palavras recebe uma palavra de 10 randomicas escolhidas de 6 ou 7
files
    db_palavras=($(shuf -n 10 "$arquivo_escolhido"))
    palavra=${db_palavras[$RANDOM % ${#db_palavras[@]}]}

    # normaliza palavra
    palavra=$(tr '[:upper:]' '[:lower:]' <<< "$palavra")

    palavra_split=$(grep -o . <<< "$palavra"))
}

## tudo front end
BOLD="\e[1m"
RED="\e[31m"
GREEN="\e[32m"
YELLOW="\e[33m"
CYAN="\e[36m"
RESET="\e[0m"

desenhar_header() {
    clear
    echo -e "${CYAN}${BOLD}===== ${RESET}"
    echo -e "                JOGO DA FORCA - SYSOPS                "
    echo -e "${CYAN}${BOLD}===== ${RESET}"
    echo -e " Tentativas restantes: ${RED}$1${RESET} | Progresso:
${GREEN}$2${RESET}"
    echo -e " Letras certas: ${GREEN}${3:-Nenhuma}${RESET}"
    echo -e " Letras erradas: ${YELLOW}${4:-Nenhuma}${RESET}"
```

```

echo -e " Letras erradas: ${YELLOW}${4:-Nenhuma}${RESET}"
echo -e " STATUS: ${YELLOW}${BOLD}$mensagem${RESET}"
echo -e "-----"
echo -e "${BOLD}Digite '!' para tentar a palavra toda.${RESET}"
echo ""
}
# uma var local para tentativa, com tratamento de entrada, apenas letras,
letra unicas e letras novas.
#letra unica e nao pode repetir
processar_tentativa() {
    local tentativa="$1"
    tentativa=$(tr '[:upper:]' '[:lower:]' <<< "$tentativa")

    if [[ ${#tentativa} -ne 1 ]]; then
        mensagem= "Digite uma letra unica."
        return 2
    fi

    if [[ ! "$tentativa" =~ ^[a-z]$ ]]; then
        mensagem= "Digite apenas letras."
        return 2
    fi

    if [[ " ${tentativas[*]} " =~ " ${tentativa} " ]]; then
        mensagem= "Você ja tentou essa letra."
        return 2
    fi

    tentativas+=("$tentativa")## update no array de Tentativas
## geral ta certo? parabens então ta certo!

    for letra in "${palavra_split[@]"; do
        if [[ "$letra" == "$tentativa" ]]; then
            mensagem= "Você acertou!"
            return 0
        fi
    done

    mensagem= "Você errou!"
    return 1
}
##mecanica do chute! Vai daqui mesmo! apenas "!" ativa isso aqui
processar_chute() {
    read -p 'Digite a palavra completa: ' chute
    chute=$(tr '[:upper:]' '[:lower:]' <<< "$chute")

    if [[ "$chute" == "$palavra" ]]; then
        desenhar_header "$tentativas_restantes" "$palavra"
        "${letras_certas[*]}" "${letras_erradas[*]}"
        mensagem= " Você acertou a palavra inteira!"
        return 0
    else
        mensagem= " Errou o chute! Fim de jogo."
    fi
}

```

```

        return 1
    fi
}
#aqui é pra atualizar o placar,
gerar_progresso() {
    local progresso=""
    for letra in "${palavra_split[@]}; do
        if [[ " ${letras_certas[*]} " =~ " ${letra} " ]]; then
            progresso+="$letra "
        else
            progresso+="_ "
        fi
    done
    echo "$progresso"
}

#INICIALIZAÇÃO
preset_jogo

# ===== LOOP Principal =====
while true; do ## esse ta aqui para garantir o rerun
    while ((tentativas_restantes > 0)); do ## esse aqui é par rodar o jogo em
    sí
        progresso=$(gerar_progresso)
        desenhar_header "$tentativas_restantes" "$progresso"
        "${letras_certas[*]}" "${letras_erradas[*]}"

        read -p 'Digite uma letra: ' tentativa

        if [[ "$tentativa" == "!" ]]; then
            processar_chute
            tentativas_restantes=0
        else
            processar_tentativa "$tentativa"
            resultado=$?

            if [[ $resultado -eq 0 ]]; then
                [[ ! " ${letras_certas[*]} " =~ " ${tentativa} " ]] &&
                letras_certas+=("$tentativa")
            elif [[ $resultado -eq 1 ]]; then
                letras_erradas+=("$tentativa")
                ((tentativas_restantes--))
            fi
        fi

        progresso=$(gerar_progresso)

        if [[ ! "$progresso" =~ "_" ]]; then
            echo " Você venceu! Palavra: $palavra"
            tentativas_restantes=0
        fi
    done
done

```

```
echo " Fim de jogo! A palavra era: $palavra"

echo ""
read -p "Bora jogar forca? (s/n): " bora
bora=$(tr '[:upper:]' '[:lower:]' <<< "$bora")

if [[ "$bora" == "s" ]]; then
    preset_jogo
    continue
else
    echo "Valeu por jogar!"
    break
fi
done
```